XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

EEE312 Week3

Lab Practice

Instructions:

- Write each exercise in to codeblocks in separate C file
- Run each program, understand the results

Exercise

```
// Write a C program to find the inverse of an NxN matrix using Gauss
Jordan elimination.

// Sample code for Lab practice
```

```
1  // inversematrix.c
2  // Find the matrix inverse of an NxN matrix
3  #include <stdio.h>
4  #define N 4
5
6  void initializeIdentityMatrix(double [][N]); // function to initialize the identity matrix
7  void rowoperation(double [][N], double [][N]); // function to perform row operation on matrix A and B
8  void backwardRowOperation(double [][N], double [][N]); // function to perform row operation backward on A and B
9  void invertMatrix(double [][N], double [][N]); // function to perform the inverse of matrix
10 void printMatrix(double [][N]); // function to print an NxN matrix
```

Function initializeIdentityMatrix()

```
32 void initializeIdentityMatrix(double B[][N])
33 {
34     for(int i=0; i<N; i++) // do for every row i=0 to N-1
35         for(int j=0; j<N; j++) // do for every column from j=0 to N-1
36         {
37             if(i==j) // if the element is diagonal i.e. i==j
38                 B[i][j] = 1.0; // initialize the element to 1
39             else
40                 B[i][j] = 0.0; // otherwise initialize the element to 0
41         }
42 }
```

Function rowoperation()

The function rowoperation is modified from the function used in last week practical on matrix determinant.

Line 55 and 56 of the code below are modified to carry out the row operation on the two matrices A and B

```
44  void rowoperation(double A[][N], double B[][N])
45  {
46      // row operations
47      double m;
48      for(int k=0; k<N-1; k++) // do for row k=0 to k=N-2; k is the row index where the pivot element is located
49      {
50          for(int i=k+1; i<N; i++) // do for row below k from k+1 to N-1; i is the row index where there is NO pivot element
51          {
52              m = A[i][k]/A[k][k]; // A[k][k] is the pivot
53              for(int j=0; j<N; j++) // For row i, perform element wise operation from j=0 to N-1. j is the column index
54              {
55                  A[i][j] -= m*A[k][j]; // At row i, perform R[i][j]<-R[i][j]-m*R[k][j] on matrix A
56                  B[i][j] -= m*B[k][j]; // At row i, perform R[i][j]<-R[i][j]-m*R[k][j] on matrix B
57              }
58          }
59      }
60  }
```

Function backeardRowOperation()

The function backwardRowOperation()is modified from the function rowoperation()from the previous slide.

Line 65 is modified such that the pivot element is chosen starting from the last row and end at the second row.

Line 67 is modified such that the row operation is carried out from the first row until the row above the row with the pivot element.

```
62  void backwardRowOperation( double A[][N], double B[][N])
63  {
64      double m = 0;
65      for(int k=N-1; k>0; k--) // start from the last row k=N-1 go backward until k=1
66      {
67          for(int i=0; i<k; i++)// do for the row i=0 until i<k where k is the row with the pivot element
68          {
69              m = A[i][k]/A[k][k]; // A[k][k] is the pivot
70              for(int j=0; j<N; j++) // For row i, perform element wise operation from j=0 to N-1. j is the column index
71              {
72                  A[i][j] = A[i][j] - m*A[k][j]; // At row i, perform R[i][j]<-R[i][j]-m*R[k][j] on matrix A
73                  B[i][j] = B[i][j] - m*B[k][j]; // At row i, perform R[i][j]<-R[i][j]-m*R[k][j] on matrix B
74              }
75          }
76      }
```

Every row I of matrix A and Bis divided with the diagonal element of matrix A i.e. A[i][i].

```
77
78      double diagonal_element = 0.0;
79      for(int i=0; i<N; i++) // do this for every row i = 0 until N-1
80      {
81          diagonal_element = A[i][i]; // store the diagonal element A[i][i] for every row i
82          for(int j=0; j<N; j++) // do this for all column from j = 0 to N-1
83          {
84              A[i][j] = A[i][j]/diagonal_element; // divide the element of every row i by the diagonal
85              B[i][j] = B[i][j]/diagonal_element; // element A[i][i] of the same row for matrix A and B
86          }
87      }
88
89  }
```

Function invertMatrix()

The function invertMatrix()is consisting of two functions: the function rowoperation() and the function backwardRowOperation().

```c
void invertMatrix(double A[][N], double B[][N])
{
    rowoperation(A, B); // carry out row operation
    backwardRowOperation(A, B); // followed by backward substitution
}
```

Function main()

```c
int main()
{
    double A[N][N] = {{8.0,2,-1,2},{-2,1,-3,-8},{2,-1,7,-1},{1,-8,1,-2}}; // left matrix A
    double B[N][N] = {{0.0}}; // right matrix B

    initializeIdentityMatrix(B);

    printf("The original matrix A is \n");
    printMatrix(A); // print the original matrix A
    printf("The original matrix B is \n");
    printMatrix(B); // print the original matrix B
    printf("\n");
    invertMatrix(A, B); // invert the matrix
    printf("The matrix A after the Gauss-Jordan elimination is\n");
    printMatrix(A); // print the matrix A (should be an NxN identity matrix)
    printf("The final inverted matrix B after the Gauss-Jordan elimination is\n");
    printMatrix(B); // print the final result of inverted matrix B
    return 0;
}
```

output

Output of the program shows the results of the matrices A and B after the Gauss-Jordan elimination.

The inverted matrix is shown as matrix B.

```
The original matrix A is
8.0000          2.0000          -1.0000         2.0000
-2.0000         1.0000          -3.0000         -8.0000
2.0000          -1.0000         7.0000          -1.0000
1.0000          -8.0000         1.0000          -2.0000

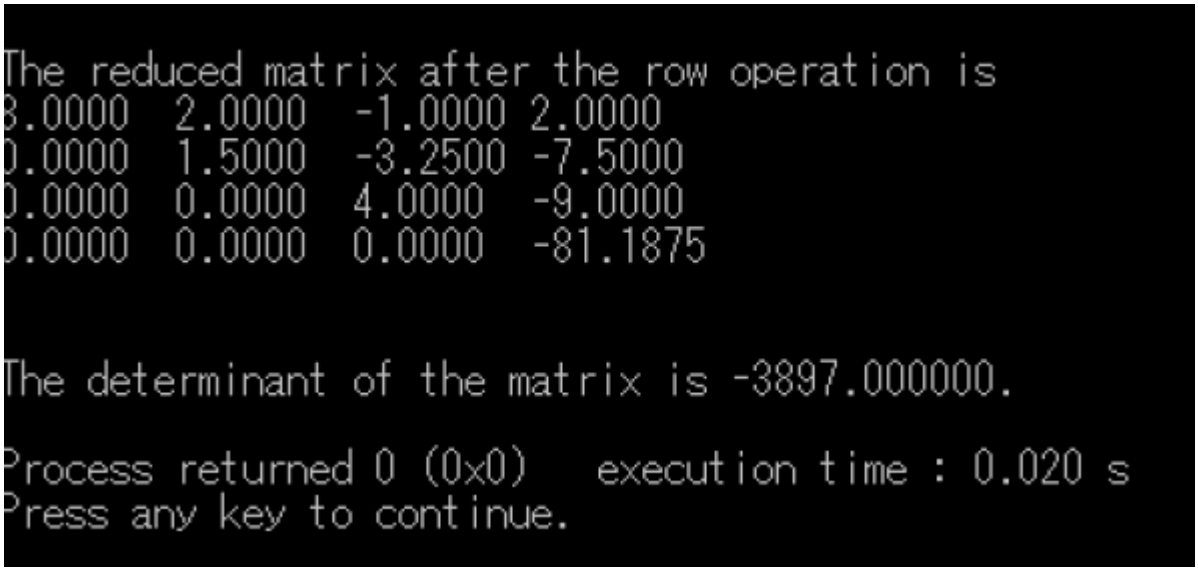
The original matrix B is
1.0000          0.0000          0.0000          0.0000
0.0000          1.0000          0.0000          0.0000
0.0000          0.0000          1.0000          0.0000
0.0000          0.0000          0.0000          1.0000


The matrix A after the Gauss-Jordan elimination is
1.0000          0.0000          0.0000          -0.0000
0.0000          1.0000          0.0000          0.0000
0.0000          0.0000          1.0000          0.0000
-0.0000         -0.0000         -0.0000         1.0000

The final inverted matrix B after the Gauss-Jordan elimination is
0.1209          0.0200          0.0216          0.0300
0.0146          0.0300          0.0323          -0.1216
-0.0346         -0.0185         0.1339          -0.0277
-0.0154         -0.1193         -0.0516         -0.0123


Process returned 0 (0x0)   execution time : 2.932 s
Press any key to continue.
```

Output


The reduced matrix after the row operation is
3.0000 2.0000 -1.0000 2.0000
0.0000 1.5000 -3.2500 -7.5000
0.0000 0.0000 4.0000 -9.0000
0.0000 0.0000 0.0000 -81.1875
The determinant of the matrix is -3897.000000.
Process returned 0 (0x0) execution time : 0.020 s
Press any key to continue.